

DevOps, Agile, fluxo e feedback

Aula 1 — do problema organizacional que originou o DevOps até o fluxo completo de entrega, operação e aprendizado.

Prof. Diogo Rodrigues



CODE



DEPLOY

PLAN

DEV

OPS



OPERATE



RELEASE



MONITOR



TEST

O que vamos abordar

01

Contexto e problema organizacional

Por que o DevOps surgiu

02

CALMS e métricas DORA

Cultura, prática e medição

03

Agile x DevOps e conceitos-chave

Pipeline, feedback, shift left

04

Ferramentas do fluxo

Código, infraestrutura e observabilidade

05

Prática em grupo

Desenhar e redesenhar o fluxo

06

Fixação e fechamento

Exercícios e próximos passos

Os sintomas do modelo tradicional

Silos e transferência de responsabilidade

"Throw over the wall"

Dev criava o código no ambiente local e "jogava" o pacote para Ops instalar em produção — daí o clássico "na minha máquina funciona, o problema é no servidor".

Processos burocráticos de liberação

Comitês e janelas raras

Como toda mudança trazia risco de queda, Ops criava comitês de aprovação rígidos (CABs) e janelas de deploy mensais ou trimestrais, de madrugada.

Tamanho das entregas

Batch size excessivo

Meses de código acumulado eram lançados de uma só vez. Quando algo falhava, descobrir qual linha ou configuração causou o incidente levava dias.

Cultura de culpa

Blamestorming

Diante de incidentes, em vez de diagnóstico rápido e aprendizado, as equipes gastavam energia apontando dedos e defendendo seus próprios silos.

2009: o ano em que o DevOps nasceu

O DevOps nasceu formalmente por volta de **2009** justamente para fundir as responsabilidades de desenvolvimento e operações.

O'Reilly Velocity 2009

Apresentação de **John Allspaw** e **Paul Hammond**: dez ou mais deploys por dia com cooperação entre Dev e Ops.

Primeiro DevOpsDays

Criado por **Patrick Debois**, em Ghent (Bélgica) — o evento que deu nome e comunidade ao movimento.



A proposta: transformar **estabilidade** e **entrega contínua** em objetivos compartilhados, por meio de **automação**, **visibilidade** e **cultura colaborativa**.

O muro entre times

Desenvolvimento otimiza mudança; operações otimiza estabilidade. Com metas opostas e entrega em lotes grandes, o resultado é fila, retrabalho e ninguém dono do fluxo.

DESENVOLVIMENTO

- Quer entregar novidades
- Mede features prontas
- “Funciona na minha máquina”

MURO

OPERAÇÕES

- Quer manter o ar funcionando
- Mede disponibilidade
- “Mudança é risco”

DevOps trata o alinhamento de dev, operações, segurança e negócio como um sistema único.

A palestra que iniciou tudo



O'Reilly Velocity 2009

“10+ Deploys Per Day: Dev and Ops Cooperation at Flickr”

John Allspaw e Paul Hammond mostraram que velocidade e estabilidade não são opostos: com responsabilidade compartilhada, automação e confiança, o Flickr publicava mais de dez vezes por dia.

Assistir no YouTube →

youtube.com/watch?v=LdOe18KhtT4

Quatro resultados desejados



Entregas menores e frequentes

Lotes pequenos, risco menor por mudança



Confiabilidade

Qualidade construída no fluxo, não inspecionada no fim



Recuperação rápida

Falhar bem importa mais que nunca falhar



Aprendizado com produção

Produção é fonte de evidência, não só de incidente

Cultura, prática, ferramenta — e resultado

01

Cultura

Crenças e incentivos

Responsabilidade compartilhada, post-mortem sem culpados

02

Prática

Comportamento repetido

Revisão de código, integração contínua, testes automatizados

03

Ferramenta

Instrumento que habilita

Git, GitHub Actions, Docker, Grafana

04

Resultado

Efeito observável

Menor lead time, menor tempo de recuperação

Ferramentas habilitam práticas; práticas sustentam cultura; cultura produz resultados.

Como medir a maturidade em DevOps?

O **CALMS** é um dos frameworks fundamentais para avaliar e diagnosticar a maturidade da adoção de DevOps em uma organização.

Origem

Criado originalmente como **CAMS** por Damon Edwards e John Willis, nos primeiros anos do movimento DevOps.

Expansão

Jez Humble acrescentou o “**L**” **de Lean**, formando o acrônimo CALMS que usamos hoje.

Para que serve

É um instrumento de diagnóstico, não uma lista de compras: mostra em qual dimensão a organização está travada.

Nas próximas telas: as cinco dimensões, uma a uma.

CALMS

Cinco dimensões para diagnosticar maturidade.

C

Culture

Colaboração,
responsabilidade
compartilhada e
segurança
psicológica

A

Automation

Reduzir tarefas
manuais
repetitivas e
inconsistentes

L

Lean

Lotes
pequenos,
limitar WIP e
remover
desperdício

M

Measurement

Medir fluxo,
qualidade e
comportamento
do sistema

S

Sharing

Compartilhar
conhecimento,
resultados e
responsabilidade

As quatro métricas DORA

DORA = DevOps Research and Assessment — o grupo de pesquisa (hoje no Google Cloud) que estudou milhares de equipes e definiu estas quatro medidas. Elas são a forma concreta de praticar o **M (Measurement)** do CALMS: duas medem velocidade, duas medem estabilidade, e são sempre lidas juntas.



Velocidade

Frequência de deploy

Com que frequência mudanças chegam à produção



Velocidade

Lead time para mudanças

Do commit até rodar em produção



Estabilidade

Taxa de falha de mudanças

% de mudanças que causam degradação



Estabilidade

Tempo de recuperação

Quanto tempo para restaurar o serviço



Use as métricas para melhorar o sistema — nunca para punir indivíduos.

Agile encurta o ciclo. DevOps o amplia.

Agile aproxima necessidade e software funcionando. DevOps segue até implantação, operação e feedback real.

AGILE

ideia → planejar → desenvolver → testar → incremento

DEVOPS

ideia → desenvolver → testar → entregar → operar → observar

← aprender — o feedback de produção volta para a ideia

Uma boa receita não basta

Um restaurante não melhora apenas criando receitas. Compra de ingredientes, cozinha, salão, entrega e avaliação dos clientes precisam funcionar como um sistema.

Receita → código bem escrito

Cozinha e salão → pipeline e operação

Entrega → deploy

Avaliação do cliente → observabilidade e feedback



Vocabulário mínimo da disciplina



Pipeline

Sequência automatizada de verificações e entregas.



Feedback rápido

Descobrir problemas perto do momento em que foram criados.



Everything as Code

Configurações versionadas, revisáveis e reproduzíveis.

E o que sustenta a operação



Shift left

Antecipar testes, segurança e qualidade.



Responsabilidade compartilhada

A equipe acompanha o software até sua operação.



Observabilidade

Compreender o estado interno do sistema a partir de suas saídas.

Do commit ao aprendizado



← aprender: o que produção mostrou volta a orientar a próxima mudança

Versionamento e integração contínua

Onde a colaboração acontece e onde o pipeline começa. Clique para abrir a documentação.



Git

Controle de versão distribuído: base de todo fluxo DevOps.

git-scm.com/doc



GitHub Actions

Automação de build, testes e deploy definida em YAML no repositório.

docs.github.com/actions



GitLab CI/CD

Pipeline integrado ao repositório, muito usado em ambientes corporativos.

docs.gitlab.com/ee/ci/



Jenkins

Servidor de automação clássico, extensível por plugins.

www.jenkins.io/doc/

Empacotamento e infraestrutura como código

Reprodutibilidade: o mesmo artefato e a mesma infraestrutura em todos os ambientes.



Docker

Empacota aplicação e dependências em imagens portáteis.

docs.docker.com/



Kubernetes

Orquestra containers: escala, autorrecuperação e implantações controladas.

kubernetes.io/docs/home/



Terraform

Infraestrutura como código declarativa, versionada e revisável.

developer.hashicorp.com/terraform/docs



Ansible

Configuração automatizada de servidores sem agentes.

docs.ansible.com/

O fluxo integrado

Para enxergar a fronteira entre elas no mundo real, imagine uma obra:

**ETAPA 1**

Terraform

Compra o terreno e constrói a estrutura do prédio

Cria VMs, VPCs e firewall na nuvem.

**ETAPA 2**

Ansible

Faz o acabamento interno

Entra nas VMs, atualiza pacotes do Linux, instala drivers.

**ETAPA 3**

Docker

Constrói as caixas de mudança seladas

Empacota backend, frontend e workers de forma padronizada.

**ETAPA 4**

Kubernetes

É o síndico e a logística

Decide onde cada caixa fica, troca as danificadas e redistribui a carga.

Qualidade, segurança e observabilidade

Sem medição não há feedback; sem feedback não há aprendizado.



Prometheus

Coleta de métricas e alertas baseados em séries temporais.

prometheus.io/docs/introduction/overview/



Grafana

Painéis para visualizar métricas, logs e traces do sistema.

grafana.com/docs/



OpenTelemetry

Padrão aberto de instrumentação para traces, métricas e logs.

opentelemetry.io/docs/



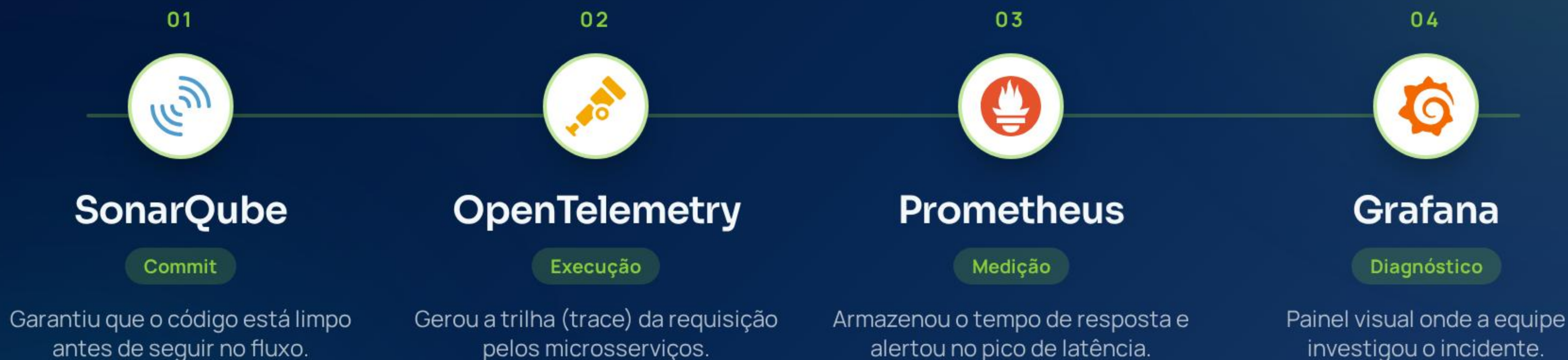
SonarQube

Análise estática de qualidade e segurança — shift left na prática.

docs.sonarsource.com/

Ciclo de vida de uma requisição

Quatro ferramentas, quatro momentos — do commit ao diagnóstico do incidente.



Qualidade no commit · rastro na execução · métrica com alerta · painel para decidir.

Quatro erros conceituais comuns

❌ “DevOps é o profissional que cuida do servidor.”

✅ O termo aparece em cargos, mas a ideia central é um modelo sociotécnico compartilhado.

❌ “Comprar ferramentas cria DevOps.”

✅ Ferramentas habilitam práticas; não corrigem silos nem incentivos ruins.

❌ “DevOps elimina operações.”

✅ Operações continua essencial — deixa de ser uma etapa isolada no fim do fluxo.

❌ “Mais deploys significam mais risco.”

✅ Mudanças pequenas, testadas e reversíveis reduzem o risco por mudança.

Três perguntas que sempre aparecem

? DevOps substitui Scrum?

Não. Scrum organiza parte do trabalho; DevOps aborda o fluxo até operação e aprendizado.

? Toda empresa precisa fazer deploy várias vezes ao dia?

Não. Frequência é contextual. O objetivo é tornar a entrega segura e disponível quando necessária.

? DevOps é igual a cloud?

Não. Cloud facilita automação, mas DevOps pode existir em infraestrutura local.

💡 **DevOps** é a cultura, o método e os processos de trabalho. A **nuvem (cloud)** é o substrato tecnológico que torna a automação e a agilidade do DevOps ordens de grandeza mais fáceis e rápidas de implementar.

Desenhe o caminho de uma mudança

Exemplo para todos os grupos: numa loja virtual, o marketing pede para mudar o botão “Finalizar compra” de azul para verde. Desenhem tudo que acontece entre o pedido e o cliente vendo o botão verde.

1. Mapear

Liste cada etapa em ordem: pedido, código, revisão, teste, aprovação, deploy, confirmação.

2. Marcar

Circule em vermelho esperas, aprovações, tarefas manuais e pontos onde ninguém sabe se deu certo.

3. Redesenhar

Refaçam o fluxo com automação e lotes menores: o que sai, o que é automatizado, o que vem antes.

Entrega do grupo: os dois desenhos + três desperdícios eliminados + tempo estimado antes e depois.

Classifique cada item

Cultura, prática, ferramenta ou resultado?

1 Revisão de código

2 GitHub Actions

3 Responsabilidade compartilhada

4 Menor tempo de recuperação

5 Docker

6 Post-mortem sem culpados

cultura

prática

ferramenta

resultado

O que levar desta aula

Fluxo

DevOps organiza o caminho da ideia até a operação — e de volta.

Feedback

Quanto mais cedo o sinal, mais barato o ajuste.

Sistema

Melhore o sistema de trabalho, não a nota individual.

Para aprofundar

Relatório DORA — DevOps Research and Assessment

dora.dev/

The Twelve-Factor App

12factor.net/pt_br/

Documentação Docker (introdução)

docs.docker.com/get-started/

CNCF Landscape — panorama de ferramentas

landscape.cncf.io/